

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэкенд-разработка

Отчёт

Домашнее задание №5
«Реализация межсервисного взаимодействия
посредством очередей сообщений»

Выполнил:
Клименков Владислав

Группа:
К3341

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Задачи

Подключить и настроить RabbitMQ для асинхронного межсервисного взаимодействия между User Service и Recipe Service. Реализовать отправку и получение событий через очередь сообщений.

Ход работы

1. Выбор архитектуры обмена сообщениями

Была выбрана модель publish/subscribe через RabbitMQ exchange:

- Exchange: `user-exchange` (тип `direct`)
- Routing keys: `user.created`, `user.deleted`
- Очередь: `recipe-service-queue` (привязана к exchange)
- Протокол: AMQP 0-9-1

User Service с помощью RabbitMQ создаёт `user.created` и `user.deleted`, а Recipe Service выступает потребителем этих сообщений.

2. Установка зависимостей

В `package.json` обоих сервисов (user-service и recipe-service) добавлена зависимость:

```
"amqplib": "^0.10.0"
```

3. Реализация Event Bus в User Service (publisher)

Создан файл `services/user/src/services/eventBus.ts`:

```
import amqp from 'amqplib';

const EXCHANGE = 'user-exchange';

let channel: amqp.Channel;

export const connectRabbitMQ = async () => {
  const connection = await
amqp.connect(process.env.RABBITMQ_URL!);
  channel = await connection.createChannel();
  await channel.assertExchange(EXCHANGE, 'direct', { durable:
```

```

true });
    console.log('[RabbitMQ] Connected and ready to publish');
};

export const publishUserCreated = async (userId: number, username:
string) => {
    if (!channel) return;
    const message = { userId, username };
    channel.publish(EXCHANGE, 'user.created',
Buffer.from(JSON.stringify(message)));
    console.log(`[RabbitMQ] Published user.created:
userId=${userId}`);
};

export const publishUserDeleted = async (userId: number) => {
    if (!channel) return;
    const message = { userId };
    channel.publish(EXCHANGE, 'user.deleted',
Buffer.from(JSON.stringify(message)));
    console.log(`[RabbitMQ] Published user.deleted:
userId=${userId}`);
};

```

Точки вызова:

1. При регистрации пользователя (`auth.service.ts:register`):

```
await publishUserCreated(newUser.id, newUser.username);
```

2. При удалении пользователя (`user.service.ts:deleteUser`):

```
await publishUserDeleted(userId);
```

4. Реализация Event Bus в Recipe Service (consumer)

Создан файл `services/recipe/src/services/eventBus.ts`:

```

import amqp from 'amqplib';
import { prisma } from '../config/database.js';

const EXCHANGE = 'user-exchange';
const QUEUE = 'recipe-service-queue';

```

```

let channel: amqp.Channel;

export const connectRabbitMQ = async () => {
  const connection = await
amqp.connect(process.env.RABBITMQ_URL!);
  channel = await connection.createChannel();

  await channel.assertExchange(EXCHANGE, 'direct', { durable:
true });
  await channel.assertQueue(Queue, { durable: true });

  // Подписка на все события пользователей
  await channel.bindQueue(Queue, EXCHANGE, 'user.created');
  await channel.bindQueue(Queue, EXCHANGE, 'user.deleted');

  await channel.consume(Queue, async (msg) => {
    if (!msg) return;

    const event = msg.content.toString();
    const routingKey = msg.fields.routingKey;
    const data = JSON.parse(event);

    console.log(`[RabbitMQ] Received event: ${routingKey} ->`,
data);

    switch (routingKey) {
      case 'user.created':
        // Пока ничего не делаем, пользователь создан в
user-service
        // Можем закешировать или создать запись при
необходимости
        break;

      case 'user.deleted':
        await handleUserDeleted(data.userId);
        break;
    }

    channel.ack(msg);
  });

  console.log('[RabbitMQ] Connected and consuming events');

```

```

};

const handleUserDeleted = async (userId: number) => {
  // Каскадное удаление всех данных пользователя в recipe-service
  await prisma.$transaction([
    prisma.commentLike.deleteMany({ where: { userId } }),
    prisma.comment.deleteMany({ where: { userId } }),
    prisma.recipeRating.deleteMany({ where: { userId } }),
    prisma.savedRecipe.deleteMany({ where: { userId } }),
    // Рецепты удаляются каскадно через связи (onDelete:
    Cascade)
    prisma.recipe.deleteMany({ where: { authorId: userId } }),
  ]);

  console.log(`[RabbitMQ] Cascaded delete for user ${userId}
  completed`);
};

```

5. Интеграция с сервисами

User Service

В `services/user/src/server.ts` добавлен вызов `connectRabbitMQ()` при запуске сервера:

```

import { connectRabbitMQ } from './services/eventBus.js';

const start = async () => {
  await connectRabbitMQ();
  app.listen(config.port, () => {
    console.log(`User Service running on port
    ${config.port}`);
  });
};

```

Recipe Service

В `**`services/recipe/src/server.ts`**` добавлен вызов `connectRabbitMQ()` при запуске сервера:

```
import { connectRabbitMQ } from './services/eventBus.js';

const start = async () => {
  await connectRabbitMQ();
  app.listen(config.port, () => {
    console.log(`Recipe Service running on port
${config.port}`);
  });
};
```

6. Конфигурация RabbitMQ

В `.env` файлы сервисов добавлены переменные окружения:

`services/user/.env`:

```
RABBITMQ_URL=amqp://guest:guest@rabbitmq:5672
```

`services/recipe/.env`:

```
RABBITMQ_URL=amqp://guest:guest@rabbitmq:5672
USER_SERVICE_URL=http://user-service:3001
```

Вывод

RabbitMQ был успешно подключён и настроен для асинхронного межсервисного взаимодействия. User Service публикует события `user.created` и `user.deleted` в exchange `user-exchange`. Recipe Service потребляет эти события через очередь `recipe-service-queue`. При получении события `user.deleted` Recipe Service каскадно удаляет все данные пользователя (рецепты, комментарии, лайки, рейтинги, сохранения). Асинхронное взаимодействие через RabbitMQ позволяет сервисам оставаться слабо связанными и не требует, чтобы оба сервиса были доступны одновременно для обработки событий.